

The Dagents Project

Data computation agents for source-level and multi-source workloads

A reusable framework for deploying one computation agent per source, combining their outputs through a global agent, and producing workload plans for real backend systems.

Source data

LMA

GMA

Framework services

Workloads

Dagents deploys computation agents near data sources, then combines their work globally.

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- One LMA is attached to each source boundary: tenant, database, service, event stream, or environment.
- Each local agent profiles data, partitions work, runs source-level models, and publishes summaries or artifacts.
- The GMA registers local agents and combines their outputs for multi-source computation.
- Core services turn those plans into workflows, model jobs, and Kubernetes workload manifests.

CMA / LMA
per source

send results

GMA
multi-source view

Some Tasks you can do...

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- **Source Profiling** - Inspecting a data source and producing a structured summary before running heavier computation.

Example:

A company has an orders table in Postgres.

Dagents can profile it and return:

- number of records
- available fields: order_id, customer_id, amount, created_at
- numeric fields: amount
- categorical fields: region, status
- missing-value checks
- partition suggestion, such as batch by created_at
- possible model tasks, such as anomaly detection on order amount

“Before running a model, Dagents first understands what kind of data it is dealing with.”

CMA / LMA
per source

send results

GMA
multi-source view

Some Tasks you can do...

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- **Pipeline Orchestration** - Defining a workflow as steps, validating the order, and running those steps.

Example Pipeline:

1. Load order records
2. Filter failed or risky transactions
3. Profile the filtered dataset
4. Route it to a suitable model
5. Save or publish the result

Dagents checks:

- Are all step IDs unique?
- Does every dependency exist?
- Is there a cycle?
- What order should the steps run in?

CMA / LMA
per source

send results

GMA
multi-source view

“Dagents makes sure the workflow is valid before executing it.”

Some Tasks you can do...

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- **Model Routing** - Choosing which model family should handle a task based on the dataset profile and requested task.

Example:

If the task is anomaly detection over tabular transaction data, Dagents may route to:

- autoencoder
- variational autoencoder

If the task is forecasting over time-series data, it may route to:

- sequence model
- forecasting model

CMA / LMA
per source

send results

GMA
multi-source view

“Instead of every app hardcoding model choices, Dagents centralizes the model-selection policy.”

Some Tasks you can do...

In the codebase, the per-source agent is called LMA (Local Monitoring Agent) or the CMA (Constraint Monitoring Agent). The outputs produced by them are combined and computations on the combined data are then done by the GMA (Global Monitoring Agent).

- **Manifest Generation-** Producing deployable Kubernetes YAML from a typed workload description.

Example input:

“We need to deploy an orders-risk-worker with 2 replicas, image orders-risk-worker:demo, port 8080, and environment variable APP_ENV=demo.”

Dagents can generate:

- Kubernetes Deployment
- Service
- ConfigMap
- sometimes Job or CronJob

“Dagents converts a clean workload description into Kubernetes deployment files.”

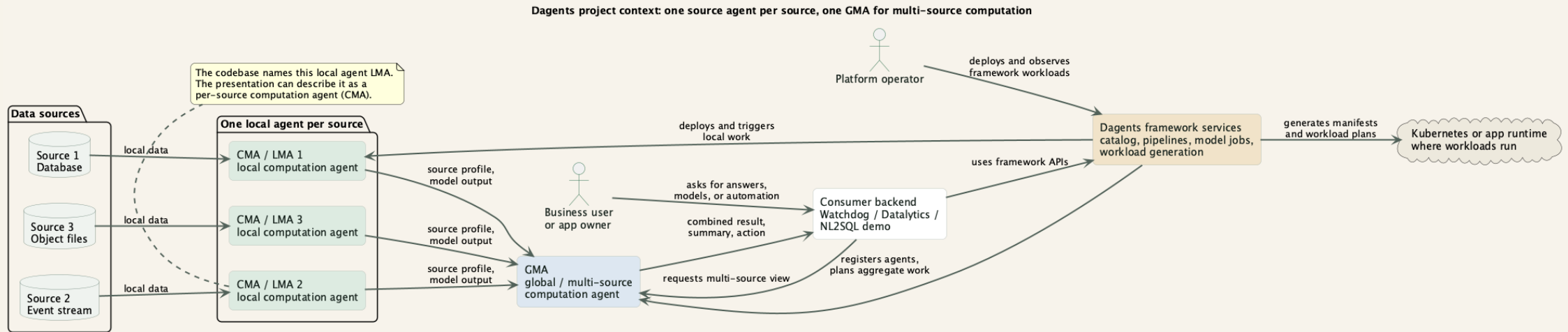
CMA / LMA
per source

send results

GMA
multi-source view

Non-technical view: local computation first, combined computation second.

The framework sits between consumer apps, data sources, operators, and the runtime where workloads are deployed.



Dagents is not one product backend. It is a reusable service framework.

Consumer apps plug into it when they need agent orchestration, ML workflow services, or workload generation.

Core service

Catalog, topology, workload compilation, and Kubernetes manifest generation.

Pipeline service

Register JSON workflows, validate them, and run them against payloads.

Model service

Python-first training, checks, benchmark data, and reusable ML jobs.

LMA / CMA

Source-scoped dataset profiling and local model execution.

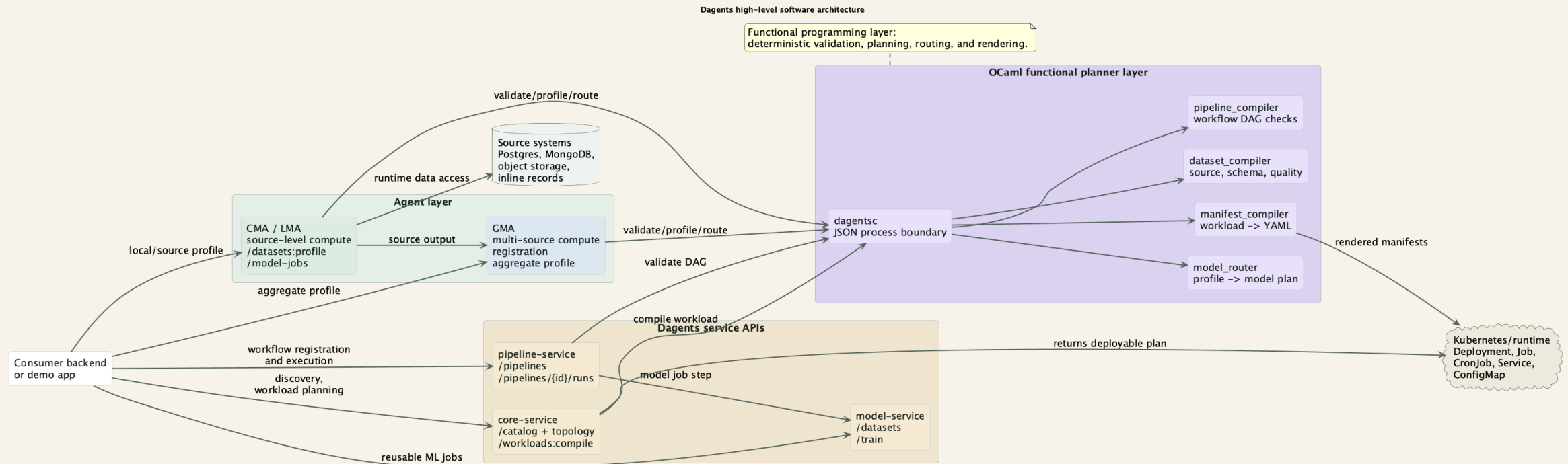
GMA

Agent registration, aggregation, dispatch tracking, and multi-source profiles.

The important idea: backend teams call stable service APIs instead of rebuilding source profiling, pipeline orchestration, model routing, and manifest generation in every app.

High-level software view: runtime services outside, functional planners inside.

The purple block is the OCaml functional programming layer. Services call it through dagentsc.



A backend integrates by calling Dagents APIs at the points where it needs planning or execution.

The system is useful because integration is service-based, not a codebase fork.

Architectural points

- LMA owns local/source execution.
- GMA owns aggregate coordination.
- Core service is the deployment facade.
- Pipeline service executes reusable workflows.
- OCaml planners keep validation deterministic.

How to integrate

- Call `/api/v1/services` or `/topology` to discover framework services.
- Register a source or pipeline definition.
- Use `LMA/GMA /datasets:profile` for local or aggregate profiles.
- Use `/workloads:compile` to turn app components into deployable manifests.
- Read returned plans and traces in your own backend/UI.

Why Dagents is split this way

The framework is intentionally polyglot: each layer owns the work it is best at.

OCaml for plans

Best for closed choices: source kinds, step kinds, model families.
Pure functions make validation repeatable and easy to test.

Python for runtime

Best for FastAPI, ML libraries, source adapters, and quick experiments.
Training and inference stay close to the Python ecosystem.

JSON process boundary

Services call dagentsc as a small planner process.
No FFI coupling; failures stay isolated and visible.

Service-based integration

Consumer backends call stable APIs instead of copying Dagents code.
The same framework can serve Watchdog, Datalytics, and NL2SQL.

Dagents is not OCaml-first or Python-first. It is planner-first where planning matters, and runtime-first where side effects matter.

Core service already exposes a deployment planning API.

This is a real FastAPI route from `services/core-service/app/api/routes.py`.

ready service API

```
@router.post("/workloads:compile", response_model=WorkloadPlanResponse)
def compile_workloads(
    request: WorkloadCompileRequest,
    service: ManifestService = Depends(get_manifest_service),
) -> WorkloadPlanResponse:
    return service.compile(request)
```

consumer call

```
curl -X POST http://127.0.0.1:8040/api/v1/workloads:compile \
-H "Content-Type: application/json" \
-d @docs/demo/inputs/core-workload-compile.json

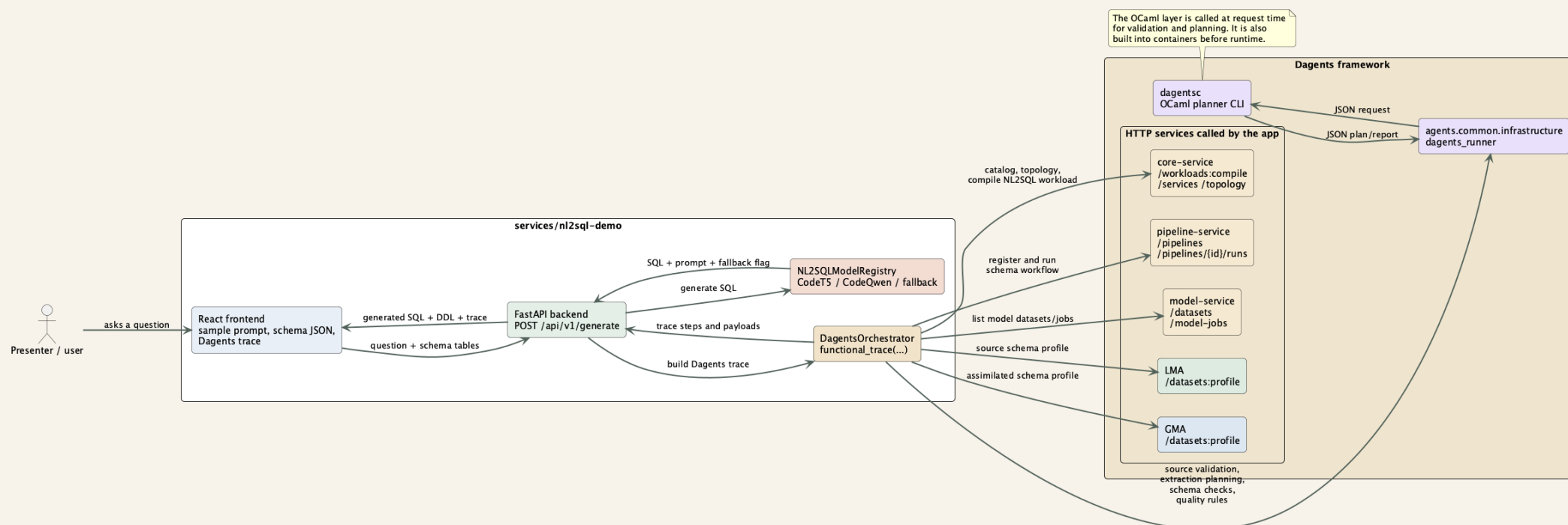
# returns a stored workload plan plus Kubernetes YAML
# for Deployment, Service, and ConfigMap resources
```

The same pattern exists across the framework: register a workflow, validate a source, profile a dataset, route a model, or compile a workload.

The demo app plugs into Dagents through a thin backend orchestration layer.

The app still owns the UI and SQL generation. Dagents owns traceable planning, validation, service checks, and workload compilation.

NL2SQL demo integration with the Dagents framework



NL2SQL calls Dagents while handling a normal /generate request.

The trace returned to the UI is built from actual planner and service calls.

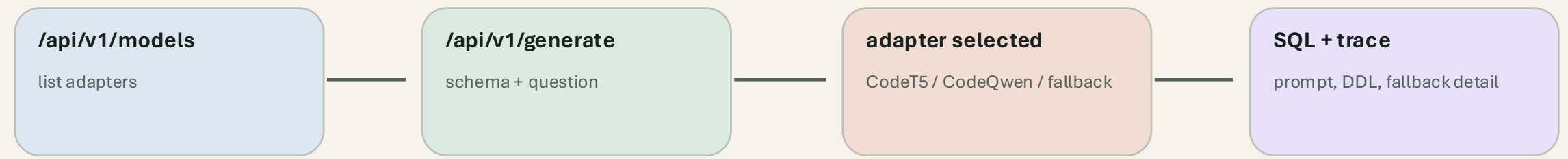
```
services/nl2sql-demo/backend/app/services/dagents_orchestrator.py
```

```
self._append_trace(trace, "Dagents SourceSpec validation",  
    lambda: validate_dataset_source(inline_source))  
self._append_trace(trace, "Dagents extraction planning",  
    lambda: compile_dataset_extraction(inline_source))  
self._append_trace(trace, "Dagents pipeline DAG planning",  
    lambda: self._pipeline_plan_payload())  
self._append_trace(trace, "core-service workload compile",  
    lambda: self._compile_demo_workload())
```

- The schema is converted into records for validation and profiling.
- dagentsc plans the schema and pipeline parts through JSON.
- Core, pipeline, model, LMA, and GMA services are touched in the trace.
- The SQL model adapter remains app-specific.

NL2SQL uses app adapters plus Dagents service calls

The app owns SQL generation; Dagents owns traceable planning, validation, and orchestration.



Adapter	What it loads	Demo behavior
CodeT5 seq2seq	Zip artifact with config and weights	Uses question/context DDL prompt for compact text-to-SQL inference.
CodeQwen LoRA	PEFT adapter over CodeQwen chat model	Uses system + user chat prompt; needs heavier runtime resources.
heuristic_demo	No model artifact	Deterministic fallback; response includes fallback_detail for clarity.

Dagents APIs touched during generation

core /services + /topology + /workloads:compile | pipeline /pipelines + /runs | LMA/GMA /datasets:profile + /model-jobs | GMA registration + heartbeat

OCaml is used where Dagents needs precise planning, not where it needs long-running services.

This keeps the functional layer small, typed, deterministic, and easy to test.

- Why: source specs, pipeline DAGs, model routes, and manifests are structured transformations.
- Where: `common_ir`, `dataset_compiler`, `pipeline_compiler`, `model_router`, and `manifest_compiler`.
- How: services call `dagentsc` with JSON and receive a plan, validation report, or YAML bundle.
- Boundary: Python and Spring still handle APIs, I/O, model training, persistence, and orchestration state.

Functional layer

compile
validate
route
render

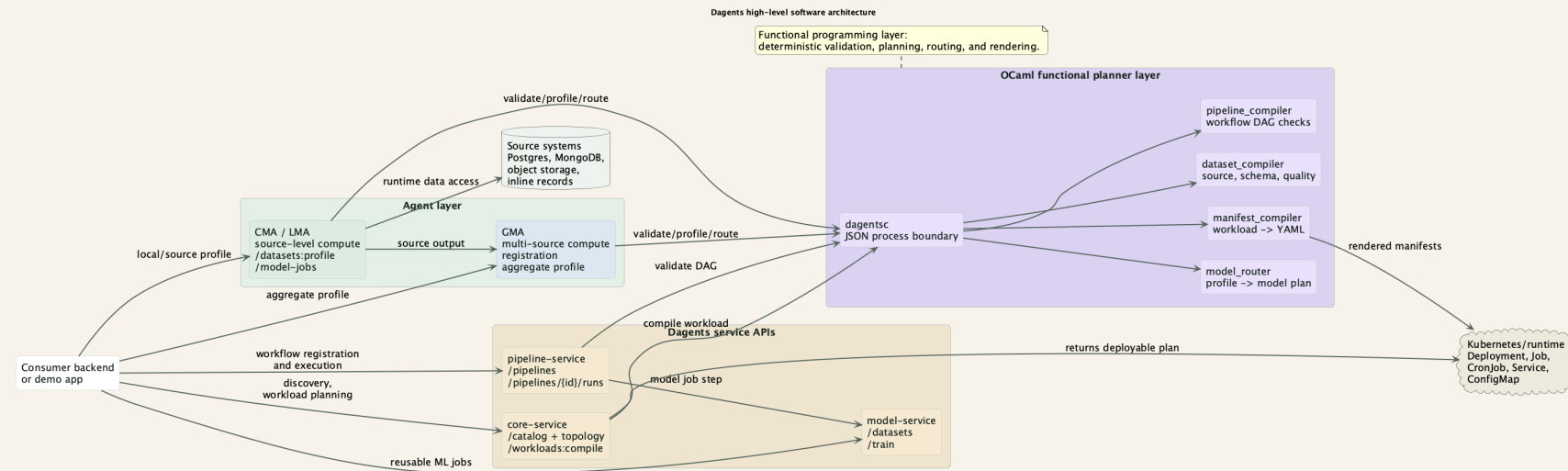
No database sockets
No training loop
No API server

In the high-level architecture, the OCaml modules are the purple planning layer.

They are called by services when a request needs deterministic validation or planning.

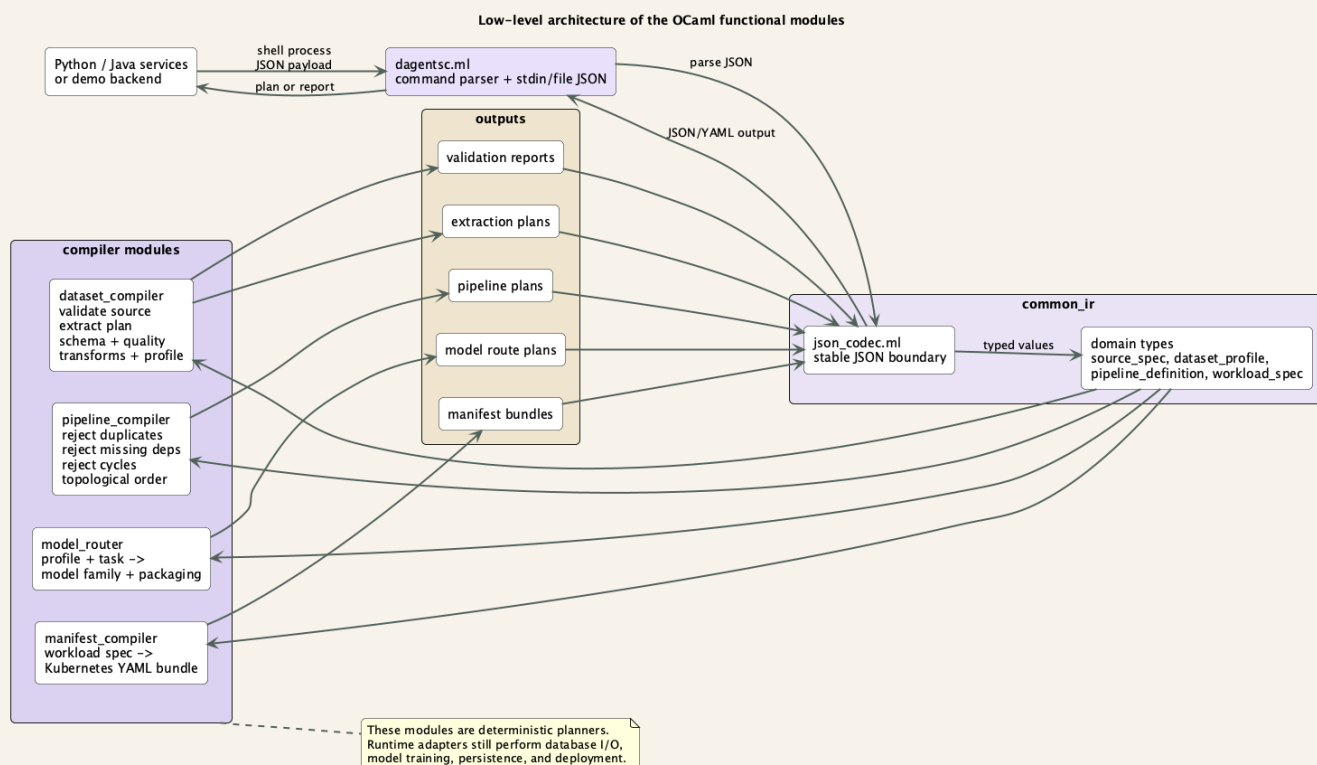
OCaml modules

- **common_ir** gives one typed language for all planners.
- **dataset_compiler** checks sources and schema work.
- **pipeline_compiler** validates workflow graphs.
- **model_router** standardizes model selection policy.
- **manifest_compiler** renders Kubernetes workload plans.



The OCaml layer is a compiler stack over shared typed contracts.

The diagram shows how services enter through dagentsc and leave with reports, plans, routes, or manifests.



- **common_ir** owns the domain shapes and JSON codecs.
- Each compiler module is a pure planning kernel.
- Outputs are explicit artifacts that runtime services can execute.
- Invalid inputs fail before the runtime does expensive work.

What each planner module does

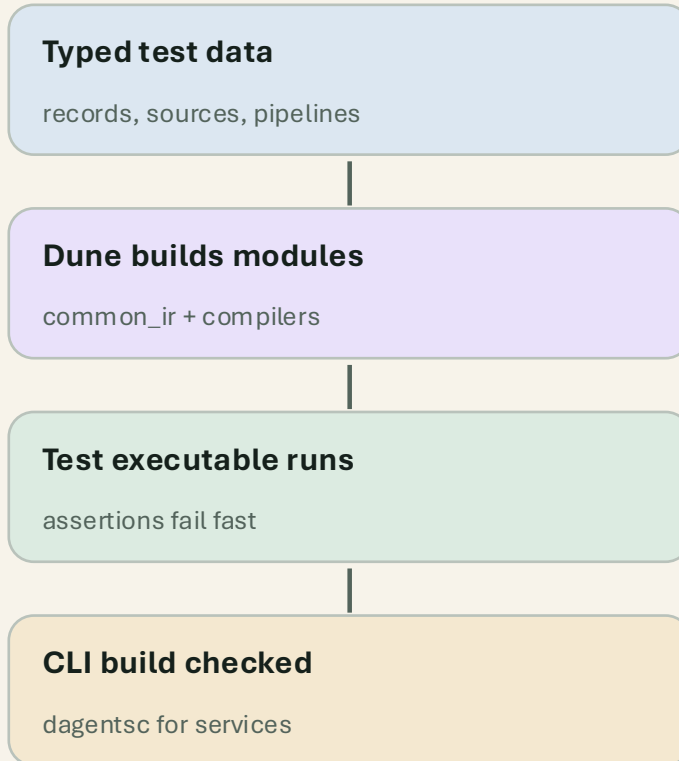
Each module has a narrow contract: typed input in, explicit artifact out.

Module	Input it understands	Output it returns	Used when
dagentsc.ml	CLI args + JSON	JSON report, plan, route, or YAML	Python/Java services need a planner call
common_ir + json_codec	Source, pipeline, profile, workload JSON	Typed records and variants	Every planner needs the same domain language
dataset_compiler	Sources, records, schema rules, transforms	Profiles, extraction plans, quality reports	Before LMA/GMA/model/pipeline data work
pipeline_compiler	Workflow DAG with dependencies	Ordered steps or validation failure	Before pipeline registration or execution
model_router	Dataset profile + task type	Candidate models + packaging mode	Before model job selection
manifest_compiler	Workload spec	Kubernetes YAML bundle	Core-service workload compilation

These are not long-running ML services. They are deterministic planning kernels that runtime services call when needed.

Dune runs the OCaml workspace tests

Dune is the OCaml build and test tool: it compiles libraries, builds dagentsc, and runs test executables.



What the tests cover

- source validation + invalid source rejection
- time-window and hash partition planning
- schema contracts and quality reports
- record transforms and output schema checks
- pipeline duplicate, missing dependency, and cycle checks
- model routing and packaging modes
- manifest rendering and JSON codec round trips

```
cd bindings/ocaml
```

```
opam exec -- dune test
```

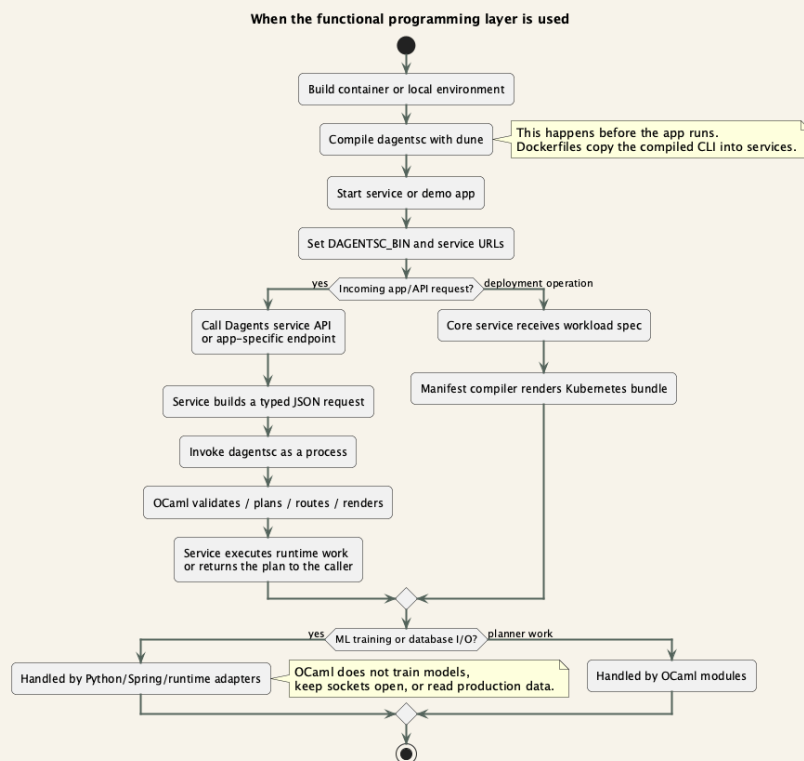
```
opam exec -- dune test -f --display=progress
```

Why this matters for the presentation

Most OCaml tests do not need Docker, databases, or GPUs because the modules are pure planners. That makes the functional layer easy to prove live.

The OCaml modules are not project ML APIs. They are planner calls used before and during runtime.

They can run before the app starts as a build artifact, and during app requests when a service needs a plan.



- Before running: Dockerfiles and scripts build dagentsc with dune.
- At startup: services receive DAGENTSC_BIN and service URLs.
- During requests: services invoke dagentsc for validation, routing, DAG planning, or manifest rendering.
- Runtime work still belongs to Python/Spring: reading data, training models, serving APIs, and storing results.

In NL2SQL, the functional layer makes the demo safer and easier to explain.

The UI can show exactly what Dagents checked and planned before returning SQL.

Useful for schema input

Source validation, extraction planning, schema contract checks, and quality rules run over the table metadata.

Useful for workflow safety

Pipeline DAG planning rejects invalid graph structure before a workflow is executed.

Useful for deployment

Core service compiles the NL2SQL backend into a workload plan and manifest bundle.

Useful for traceability

The response includes planner and service trace steps, so the demo can show what happened.

For your clarity: OCaml is called as needed by the app/services. It is not the SQL model and not a separate ML server.

Run the demo with one terminal for the app and one terminal for the probe.

Use local mode for speed. Use compose mode when you want to show the whole Dagents stack.

local mode

```
# Fast local demo
services/nl2sql-demo/scripts/run_local_demo.sh

# Probe the backend and save the response
services/nl2sql-demo/scripts/probe_api.sh
```

compose and functional demo

```
# Full Dagents stack with NL2SQL
services/nl2sql-demo/scripts/run_compose_demo.sh

# Stop containers
services/nl2sql-demo/scripts/stop_compose_demo.sh

# Functional layer output screenshots
docs/demo/run_functional_layer_demo.sh
```

Open <http://127.0.0.1:5173>

Dagents gives consumer backends reusable agent orchestration plus deterministic planning.

The project story is simple: compute locally, combine globally, plan with typed functional modules, and deploy through services.

- CMA / LMA per source keeps local computation close to the data.
- GMA combines many local outputs into multi-source computation.
- Framework services expose catalog, pipeline, model, agent, and workload APIs.
- OCaml planners make validation and planning predictable.
- NL2SQL proves a real app can plug into the framework without becoming the framework.